

≡ SPRING BOOT HANDBOOK ≡

BOOK - PART 1

1. WHAT IS SPRING & SPRING BOOT ?

⇒ What is Spring ?

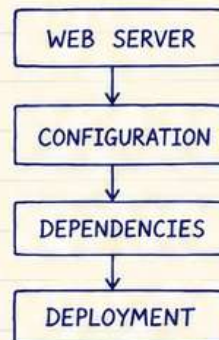
- Spring is a lightweight Java framework.
- It provides a comprehensive programming and configuration model for modern Java based enterprise applications.
- It is built on core concepts like IoC, DI and AOP.

Why Spring Boot ?

- Faster development
- Auto configuration
- Embedded servers
- Production ready
- Less XML, more Annotations
- Easy Dependency Management
- Built-in Actuator & Monitoring

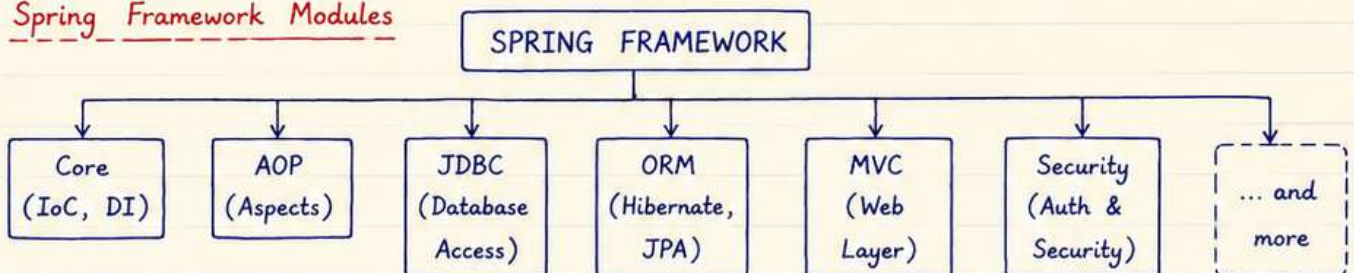
⇒ What is Spring Boot ?

- Spring Boot is an extension of Spring framework.
- It simplifies development by auto-configuration and convention over configuration.
- It helps to create stand-alone, production ready applications with minimal setup.
- It has embedded servers (Tomcat, Jetty, Undertow) and reduces boilerplate code.



Spring Boot makes it simple and ready to run.

Spring Framework Modules



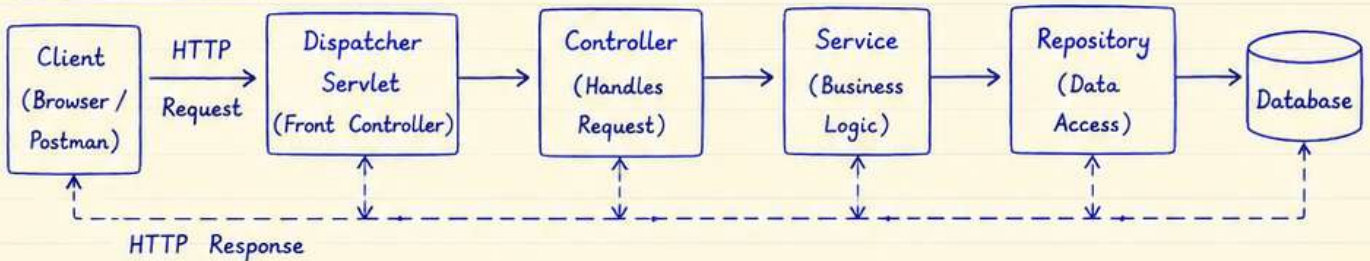
Spring vs Spring Boot

Feature	Spring Framework	Spring Boot
Configuration	Manual configuration	Auto configuration
Setup	Complex	Simple
Server	External (Tomcat, Jetty etc.)	Embedded (Tomcat, Jetty etc.)
Deployment	WAR file	JAR file
Development	Time consuming	Very fast
XML Usage	More XML	Less / No XML
Monitoring	Manual setup	Built-in Actuator
Main Goal	Provide full control	Convention over configuration

2. SPRING BOOT ARCHITECTURE

Spring Boot follows a layered architecture. Each layer has a specific responsibility and they work together to handle a request.

Architecture Flow :



Components Description :

- Client : Sends HTTP request and receives response.
- DispatcherServlet : Front controller of Spring MVC. It receives the request and dispatches it to the appropriate controller.
- Controller : Handles the request, validates input and returns response.
- Service : Contains business logic and calls repository.
- Repository : Interacts with database to perform CRUD operations.
- Database : Stores application data.

Example Flow :

Client → DispatcherServlet → Controller → Service → Repository → Database
 Database → Repository → Service → Controller → DispatcherServlet → Client

Example Code (Controller) :

```

@RestController
@RequestMapping("/api/users")
public class UserController {
    @Autowired
    private UserService userService;

    @GetMapping("/{id}")
    public User getUser(@PathVariable Long id) {
        return userService.getUser(id);
    }
}
  
```

- Marks this class as REST Controller
- Base URL for all APIs in this controller
- Injects UserService dependency
- Maps GET request with user id
- Calls service layer and returns data

Example (application.properties) :

```

server.port = 8080
spring.datasource.url = jdbc:mysql://localhost:3306/demo
spring.datasource.username = root
spring.datasource.password = root
spring.jpa.hibernate.ddl-auto = update
spring.jpa.show-sql = true
  
```

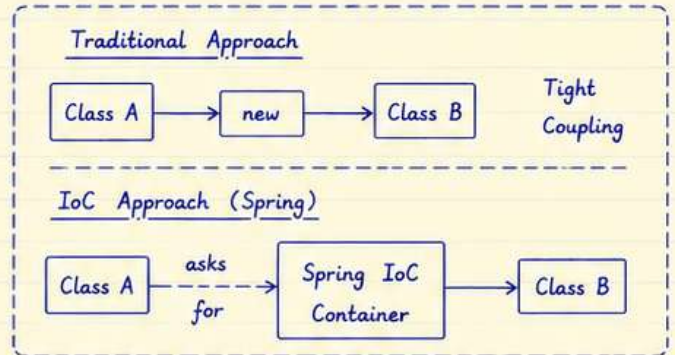
} Basic configuration
for database & server

This flow is the backbone of any Spring Boot web application.

3. IOC CONTAINER & DEPENDENCY INJECTION

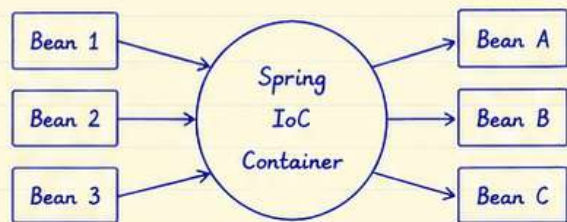
3.1 What is IoC ?

- IoC (Inversion of Control) is a principle where the control of object creation and management is transferred from application code to the Spring container.
- The container creates objects, wires them together, manages their lifecycle and dependencies.



3.2 Spring IoC Container

- Spring IoC container is responsible for creating, configuring and assembling the beans.
- The container provides the beans to the application when required.



3.3 Dependency Injection (DI)

- DI is a design pattern where the container injects the required dependencies into the objects.
- It helps in achieving loose coupling and easy unit testing.

Types of DI

- Constructor Injection
- Setter Injection
- Field Injection

Example (Constructor Injection)

```
public class UserService {
    private UserRepository userRepository;

    public UserService(UserRepository repo) {
        this.userRepository = repo;
    }
}
```

3.4 Example Code

1. Bean Class

```
@Repository
public class UserRepository {
    public String getUserById(int id) {
        return "User" + id;
    }
}

@Service
public class UserService {
    private UserRepository userRepository;

    // Constructor Injection
    public UserService(UserRepository repo) {
        this.userRepository = repo;
    }

    public String getUser(int id) {
        return userRepository.getUserById(id);
    }
}
```

2. Configuration (Java Based)

```
@Configuration
public class AppConfig {

    @Bean
    public UserRepository userRepository() {
        return new UserRepository();
    }

    @Bean
    public UserService userService(
        UserRepository userRepository) {
        return new UserService(userRepository);
    }
}
```

3. Main Class

```
public class MainApplication {
    public static void main(String[] args) {
        AnnotationConfigApplicationContext
            context = new
            AnnotationConfigApplicationContext
            (AppConfig.class);

        UserService service =
            context.getBean(UserService.class);

        System.out.println(service.getUser(1));

        context.close();
    }
}
```

3.5 Key Points

- IoC reduces tight coupling between classes.
- DI makes code more testable and maintainable.
- Spring container manages the lifecycle of beans.
- We just declare our dependencies, container takes care of creation and injection.

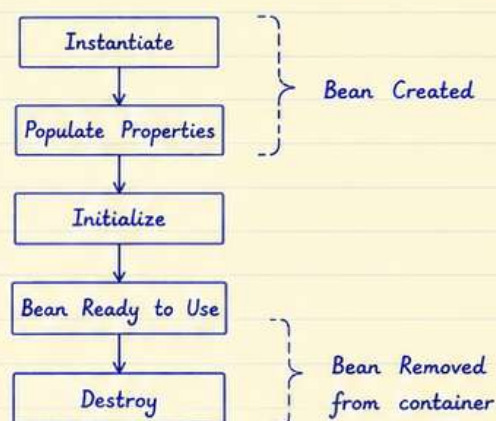
4. BEAN LIFECYCLE & BEAN SCOPE

4.1 Bean Lifecycle

Spring manages the complete lifecycle of a bean.

The steps are :

- ① Instantiate → Container creates bean instance.
- ② Populate → Set values to the bean properties.
- ③ Initialize → Call `BeanNameAware`, `InitializingBean`, `@PostConstruct` etc.
- ④ Ready → Bean is ready to use.
- ⑤ Destroy → Call `@PreDestroy`, `DisposableBean` before removing bean.



4.2 Example of Lifecycle

```

public class DemoBean implements InitializingBean, DisposableBean {
    public DemoBean() {
        System.out.println("1. Constructor");
    }

    @Override
    public void afterPropertiesSet() {
        System.out.println("2. afterPropertiesSet()");
    }

    @PreDestroy
    public void destroy() {
        System.out.println("5. @PreDestroy");
    }
}
  
```

Output Order :

1. Constructor
2. `afterPropertiesSet()`
3. Bean is ready to use
4. Bean will do its work
5. `@PreDestroy`

Note : Steps 1 and 2 are done by Spring container for every bean.

4.3 Bean Scopes

Scope	Description	Where it is used	Annotation / Config
Singleton	Only one instance per Spring IoC container.	Default scope.	<code>@Scope("singleton")</code> (default)
Prototype	New instance for every request.	When each request needs separate object.	<code>@Scope("prototype")</code>
Request	One instance per HTTP request.	Web applications.	<code>@Scope("request")</code>
Session	One instance per HTTP session.	Web applications.	<code>@Scope("session")</code>
Application	One instance per ServletContext.	Web applications.	<code>@Scope("application")</code>
Websocket	One instance per WebSocket session.	WebSocket applications.	<code>@Scope("websocket")</code>

Example :

```

@Component
@Scope("prototype")
public class Student {
    // class body
}
  
```

→ Every time container is asked for Student bean, it will create new object.

5. SPRING ANNOTATIONS

5.1 Common Stereotype Annotations

Annotation	Used For	Description	Example
@Component	General component	Marks a class as a Spring managed component.	@Component public class EmailUtil { }
@Service	Service layer	Indicates that the class belongs to service layer.	@Service public class UserService { }
@Repository	Data access layer	Indicates that the class is responsible for database access.	@Repository public interface UserRepository extends JpaRepository<User, Long>{ }
@Controller	Web MVC controller	Marks a class as Spring MVC Controller. Returns view name.	@Controller public class HomeController { }
@RestController	REST API controller	Combination of @Controller and @ResponseBody.	@RestController public class UserController { }
@Configuration	Java based configuration	Indicates that the class provides bean definitions.	@Configuration public class AppConfig { }
@Bean	Bean definition method	Marks a method as a bean creator in @Configuration class.	@Bean public DataSource dataSource(){ return new HikariDataSource(); }

5.2 AutoWiring

- AutoWiring is used to inject the bean dependencies automatically.
- Types : byType (default), byName, constructor, no

```
@Service
public class UserService {
    // dependency
    @Autowired           // byType (default)
    private UserRepository userRepository;
}
```

Other ways :

- byName → matches by property name
- constructor → injects through constructor
- no → no autowiring

5.3 Component Scan

- Spring Boot automatically scans the packages under the main class package and its sub-packages.
- It detects @Component, @Service, @Repository, @Controller etc. and registers them as beans.

```
@SpringBootApplication
public class DemoApplication {
    public static void main(String[] args) {
        SpringApplication.run(DemoApplication.class, args);
    }
}
```

→ This annotation includes
@ComponentScan, @Configuration
and @EnableAutoConfiguration.

5.4 Other Important Annotations

- @Value → Injects values from application.properties / application.yml
- @Qualifier → Used with @Autowired to resolve ambiguity.
- @Primary → Marks one bean as primary when multiple beans of same type exist.
- @Lazy → Initializes the bean only when it is required.

6. REST API BASICS IN SPRING BOOT

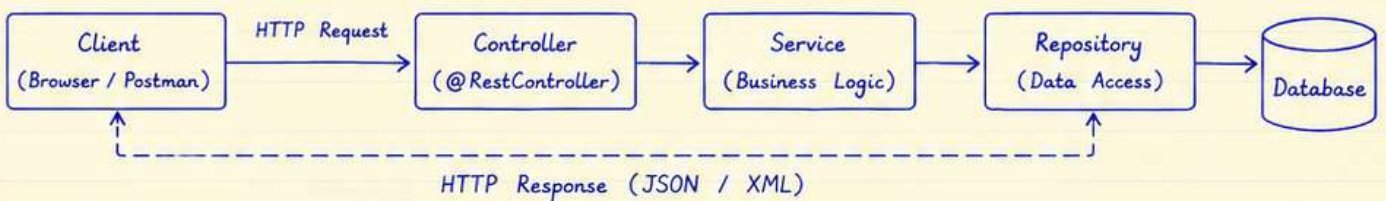
6.1 What is REST API ?

- REST (Representational State Transfer) is an architectural style for designing networked applications.
- It uses HTTP methods to perform CRUD operations on resources.
- Spring Boot makes it very easy to build RESTful web services.

CRUD using HTTP Methods

HTTP Method	Operation	Description
GET	Read	Fetch data
POST	Create	Create new resource
PUT	Update	Update entire resource
PATCH	Partial Update	Update partial data
DELETE	Delete	Delete resource

6.2 REST API Flow



6.3 Example - Simple REST Controller

```

@RestController
@RequestMapping("/api/users")
public class UserController {

    @GetMapping("/{id}")
    public User getUser(@PathVariable Long id) {
        return new User(id, "Aman", "aman@example.com");
    }

    @PostMapping
    public User createUser(@RequestBody User user) {
        user.setId(101L);
        return user;
    }

    @PutMapping("/{id}")
    public User updateUser(@PathVariable Long id,
                           @RequestBody User user) {
        user.setId(id);
        return user;
    }

    @DeleteMapping("/{id}")
    public String deleteUser(@PathVariable Long id) {
        return "User deleted with id: " + id;
    }
}
  
```

Explanation :

- @RestController = @Controller + @ResponseBody
- @RequestMapping defines base URL for the class
- @GetMapping handles GET request
- @PostMapping handles POST request
- @PutMapping handles PUT request
- @DeleteMapping handles DELETE request
- @PathVariable fetches value from URL
- @RequestBody binds request body to object

Sample JSON (Request Body)

```

{
  "name": "Aman",
  "email": "aman@example.com",
  "age": 25
}
  
```

6.4 Response Example

GET /api/users/1 → Response (200 OK)

```

{
  "id": 1,
  "name": "Aman",
  "email": "aman@example.com"
}
  
```

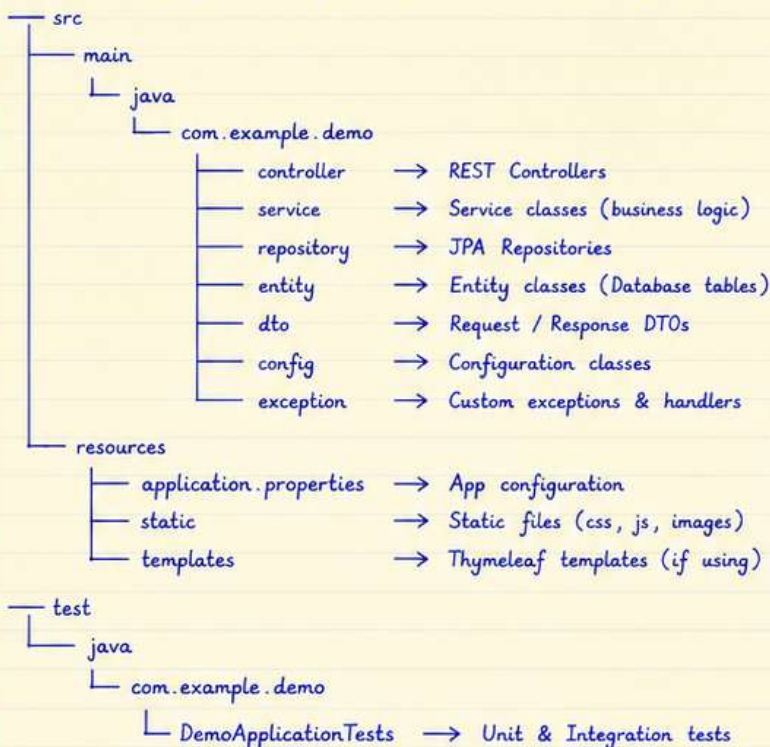
POST /api/users → Response (201 Created)

```

{
  "id": 101,
  "name": "Aman",
  "email": "aman@example.com"
}
  
```

7. SPRING BOOT PROJECT STRUCTURE

A typical Spring Boot project follows a standard folder structure.



Important Files

- DemoApplication.java → Main class
- application.properties → Configuration
- pom.xml → Dependencies
- build.gradle → Build file

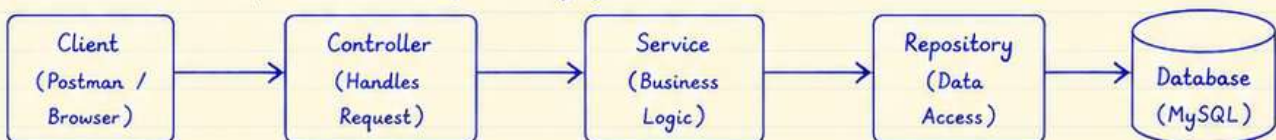
Main Class Example

```

@SpringBootApplication
public class DemoApplication {
    public static void main(String[] args) {
        SpringApplication.run(DemoApplication.class, args);
    }
}
  
```

8. BUILD FIRST CRUD API - OVERVIEW FLOW

We will build a simple CRUD API for managing Student data.



CRUD Endpoints

- Create - POST /api/students
- Read All - GET /api/students
- Read By Id - GET /api/students/{id}
- Update - PUT /api/students/{id}
- Delete - DELETE /api/students/{id}

Base URL

http://localhost:8080/api/students

Technology Used

- Spring Boot
- Spring Web
- Spring Data JPA
- MySQL
- Lombok (optional)
- Validation
- Postman (for testing)

Dependencies in pom.xml

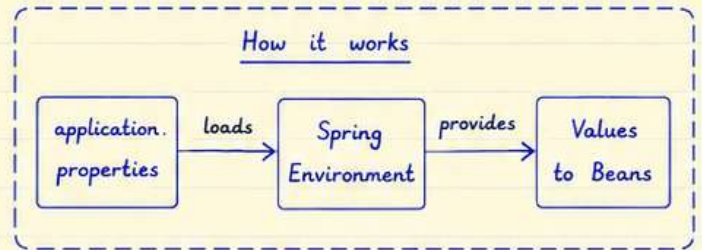
```

<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-web</artifactId>
</dependency>
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>
<dependency>
  <groupId>mysql</groupId>
  <artifactId>mysql-connector-java</artifactId>
  <scope>runtime</scope>
</dependency>
  
```

9. APPLICATION PROPERTIES & CONFIGURATION

9.1 What is application.properties ?

- It is the main configuration file in Spring Boot. We use it to configure database, server port, logging, and other properties.



9.2 Where is it located ?

- src/main/resources/application.properties

9.3 Example Properties

```

server.port = 8080

spring.application.name = demo-app

spring.datasource.url = jdbc:mysql://localhost:3306/demo
spring.datasource.username = root
spring.datasource.password = root123
spring.jpa.hibernate.ddl-auto = update
spring.jpa.show-sql = true

logging.level.org.springframework = INFO
logging.level.com.example.demo = DEBUG
  
```

9.4 Accessing Properties in Code

Using @Value

```

@Value("${app.name}")
private String appName;

@Value("${server.port}")
private int port;
  
```

Using @ConfigurationProperties

```

@Component
@ConfigurationProperties(prefix = "app")
public class AppProperties {
    private String name;
    private String description;
    // getters and setters
}
  
```

9.5 Custom Properties Example

application.properties

```

app.name = Student Service
app.description = This is a demo application
  
```



```

@Component
@ConfigurationProperties(prefix = "app")
public class AppProperties {
    private String name;
    private String description;
    // getters and setters
}
  
```

9.6 Profile based Properties

We can have different properties for different environments.

```

├── resources
│   ├── application.properties      → (default)
│   ├── application-dev.properties → (for dev environment)
│   └── application-prod.properties → (for prod environment)
  
```

Activate Profile

In application.properties
 spring.profiles.active = dev
 (or)

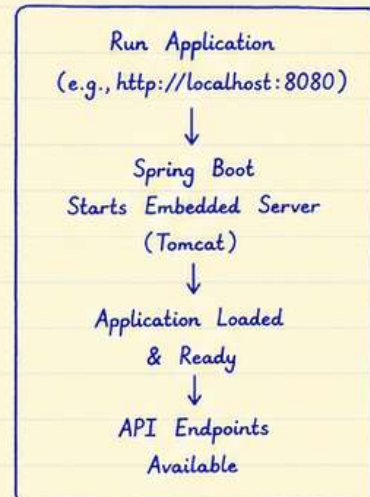
Run from command line
 --spring.profiles.active = dev

10. SPRING BOOT RUN & TEST

10.1 Run the Application

- We can run the Spring Boot application in multiple ways.

Method	Command
1. From IDE	Right click on DemoApplication.java → Run 'DemoApplication'
2. Using Maven	mvn spring-boot:run
3. Using Gradle	gradle bootRun
4. Using JAR	mvn clean package java -jar target/demo-app.jar



10.2 Sample API Endpoints (from our project)

Method	Endpoint	Description	Sample Response
GET	/api/students	Get all students	[{"id": 1, "name": "Aman"}, ...]
GET	/api/students/1	Get student by id	{"id": 1, "name": "Aman", "age": 25}
POST	/api/students	Create new student	{"id": 101, "name": "Aman", "age": 25}
PUT	/api/students/1	Update student	{"id": 1, "name": "Aman", "age": 26}
DELETE	/api/students/1	Delete student	{"message": "User deleted"}

10.3 Testing the APIs

- We can test APIs using Postman / Browser / cURL

Example : GET /api/students/1

Request (GET)

http://localhost:8080/api/students/1

Response (200 OK)

```
{
  "id": 1,
  "name": "Aman",
  "email": "aman@example.com",
  "age": 25
}
```

Example cURL (GET)

```
curl -X GET http://localhost:8080/api/students/1
-H "Accept: application/json"
```

Example cURL (POST)

```
curl -X POST http://localhost:8080/api/students
-H "Content-Type: application/json"
-d '{"name": "Aman", "email": "aman@example.com", "age": 25}'
```

10.4 Common Application Properties

Property	Use
server.port = 8080	Change application port
spring.application.name = demo-app	Set application name
spring.datasource.url = jdbc:mysql://localhost:3306/demo	Database connection URL
spring.datasource.username = root	Database username
spring.datasource.password = 123	Database password
spring.jpa.hibernate.ddl-auto = update	Auto create / update tables
logging.level.org.springframework = INFO	Set logging level

What we covered in Part - 1

1. Spring Boot Overview
2. Project Structure
3. Maven Dependencies
4. Bean Lifecycle & Scope
5. Spring Annotations
6. REST API Basics
7. Project Structure & CRUD Flow
8. Application Properties
9. Run & Test Application

Next → Advanced Topics in Part - 2

FOLLOW FOR 
PART 2

COMMENT 
SPRING BOOT

♥ FOR COMPLETE ♥

PDF NOTES OF PART 1



SAVE



SHARE



FOLLOW



Aman Views



SPRING BOOT HANDBOOK

PART - 2

Page - 1/11

1. SPRING DATA JPA

- Spring Data JPA is a part of Spring Data project.
- It makes it easy to implement JPA based repositories.
- It reduces the amount of boilerplate code required to access the data.
- It provides CRUD operations, pagination, sorting and more with very less code.

HOW SPRING DATA JPA WORKS ?



KEY FEATURES

- Repository abstraction
- Automatic implementation of repository interfaces
- Supports derived query methods
- Pagination and sorting
- Custom queries using @Query
- Auditing, Specifications, Projections and more

DEPENDENCIES (MAVEN)

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>

<dependency>
  <groupId>mysql</groupId>
  <artifactId>mysql-connector-java</artifactId>
  <scope>runtime</scope>
</dependency>
```

* mysql dependency is optional, based on the database you are using.

ENTITY EXAMPLE

```
@Entity
@Table(name="users")
public class User {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String name;
    private String email;
}
```

REPOSITORY EXAMPLE

```
import org.springframework.data.jpa.repository.
JpaRepository;

public interface UserRepository
    extends JpaRepository<User, Long> {
}
```

- By extending JpaRepository, we get all CRUD methods.
- No need to write implementation class.

SPRING BOOT HANDBOOK

PART - 2

Page - 2/11

2. ENTITY MAPPING (JPA)

- Entity mapping is the process of mapping Java classes to database tables.
- JPA (Jakarta Persistence API) provides annotations to define mappings.
- The `@Entity` annotation marks a class as a JPA entity.
- Each entity must have a primary key.
- Relationships can be mapped between entities.

COMMON ANNOTATIONS

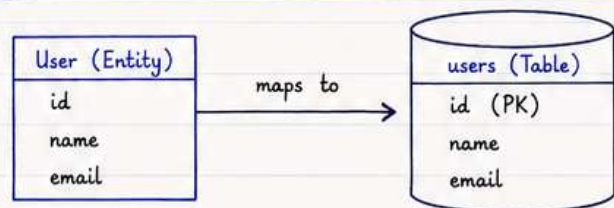
Annotation	Description
<code>@Entity</code>	Marks the class as an entity
<code>@Table</code>	Specifies the table name
<code>@Id</code>	Marks the primary key
<code>@GeneratedValue</code>	Strategy for primary key generation
<code>@Column</code>	Maps a field to a column
<code>@OneToOne</code>	One-to-one relationship
<code>@OneToMany</code>	One-to-many relationship
<code>@ManyToOne</code>	Many-to-one relationship
<code>@ManyToMany</code>	Many-to-many relationship

EXAMPLE

```
@Entity
@Table(name = "users")
public class User {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(name = "name", nullable = false)
    private String name;

    @Column(name = "email", unique = true)
    private String email;
}
```



3. REPOSITORY INTERFACES

- Spring Data JPA provides Repository interfaces to perform CRUD operations.
- These interfaces are part of the `org.springframework.data.jpa.repository` package.
- No need to write implementation classes.
- Query methods are automatically implemented based on method names.
- Supports pagination, sorting and custom queries.

COMMON REPOSITORY INTERFACES

Interface	Description
<code>CrudRepository<T, ID></code>	Basic CRUD operations
<code>PagingAndSortingRepository<T, ID></code>	CRUD + Pagination and Sorting
<code>JpaRepository<T, ID></code>	CRUD + Pagination + Sorting + JPA specific methods

EXAMPLE

```
import org.springframework.data.jpa.repository.JpaRepository;

public interface UserRepository extends JpaRepository<User, Long> {
    // Query method - will be implemented automatically
    List<User> findByName(String name);
    User findByEmail(String email);
    List<User> findByNameContaining(String keyword);
}
```

HOW IT WORKS ?



≡ SPRING BOOT HANDBOOK ≡

PART - 2

Page - 3/11

3. SPRING DATA JPA

- Spring Data JPA is a part of Spring Data project.
- It provides easy and powerful way to perform CRUD operations on database.
- It is built on top of JPA (Hibernate) and simplifies data access layer.
- It reduces boilerplate code for repository implementations.
- It follows repository abstraction and provides ready-made methods.

HOW IT WORKS ?

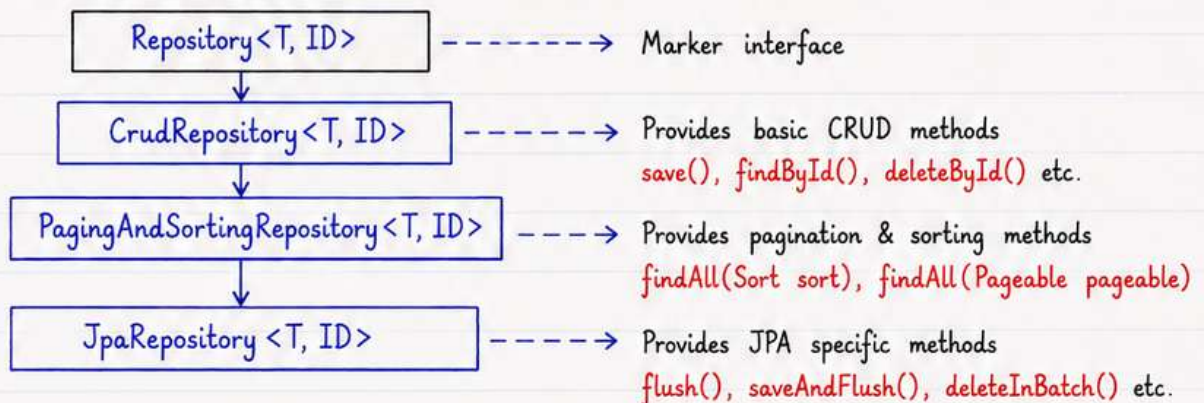


- * We just need to create an interface that extends JpaRepository. Spring Data JPA will provide implementation at runtime.

KEY FEATURES

- Provides repository abstraction.
- Supports pagination, sorting.
- Supports derived query methods.
- Custom query support using @Query.
- Auditing support.
- Easy integration with Spring Boot.

JPA REPOSITORY HIERARCHY



EXAMPLE - REPOSITORY INTERFACE

```
import org.springframework.data.jpa.repository.JpaRepository;

public interface UserRepository extends JpaRepository<User, Long> {

    List<User> findByName(String name);

    List<User> findByEmailContaining(String email);

}
```

- Entity class → User
- Primary Key Type → Long
- Derived query method
- Derived query method

SPRING BOOT HANDBOOK

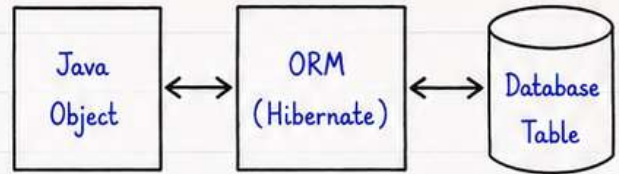
PART - 2

Page - 4/11

4. HIBERNATE & ORM

- Hibernate is the default JPA implementation used in Spring Boot.
- It provides ORM (Object Relational Mapping) between Java Objects and Database tables.
- It handles SQL generation, mapping, caching, transactions etc.
- It reduces boilerplate JDBC code.
- We just focus on entity classes, Hibernate takes care of the rest.

HOW ORM WORKS ?



- * ORM maps Java classes to database tables and Java objects to table rows.
- We perform operations on objects, Hibernate converts them into SQL queries.

ENTITY ANNOTATIONS

Annotation	Use
@Entity	Marks a class as an entity
@Table	Specifies the table name
@Id	Marks the primary key
@GeneratedValue	Auto generates primary key value
@Column	Specifies column details
@Temporal	Specifies date/time type
@Enumerated	Used with Enum types
@Lob	Used for large objects (BLOB,CLOB)

ENTITY EXAMPLE

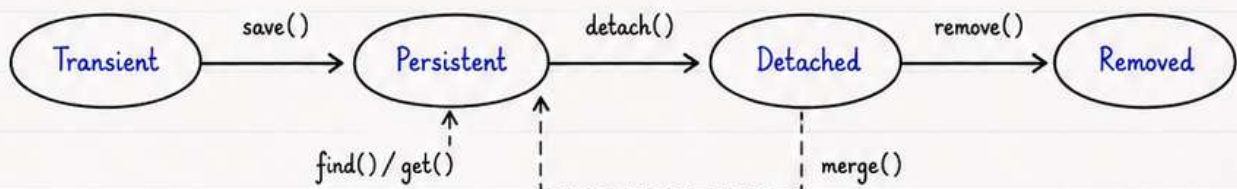
```
@Entity
@Table(name = "users")
public class User {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(name = "name", nullable = false, length = 100)
    private String name;

    @Column(name = "email", unique = true, nullable = false)
    private String email;

    @Temporal(TemporalType.TIMESTAMP)
    private Date createdAt;
}
```

ENTITY STATES



JPA INHERITANCE STRATEGIES

Strategy	Description	Use
SINGLE_TABLE	All classes in a single table with a discriminator column	Best performance
JOINED	Each class has its own table, joined by foreign key	Normalized structure
TABLE_PER_CLASS	Each class has its own table	Less common

Note:

Hibernate is powerful but we should always understand the SQL queries it generates.

≡ SPRING BOOT HANDBOOK ≡

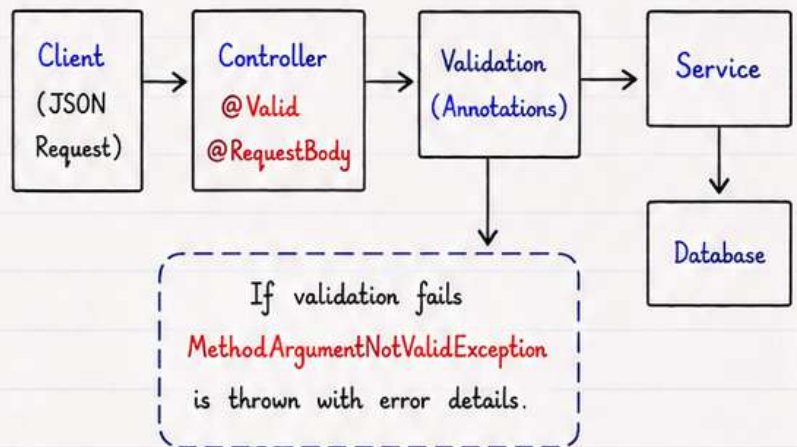
PART - 2

Page - 5/11

5. VALIDATION

HOW IT WORKS ?

- Validation ensures that the data coming from user is correct and in the expected format.
- Spring Boot provides validation using Jakarta Bean Validation API.
- We use validation annotations on fields and `@Valid` on request body.
- If validation fails, Spring Boot throws `MethodArgumentNotValidException`.



COMMON VALIDATION ANNOTATIONS

Annotation	Use
<code>@NotNull</code>	Field must not be null
<code>@NotBlank</code>	String must not be null or empty
<code>@Size(min, max)</code>	String/Collection size must be between min and max
<code>@Min(value)</code>	Number must be greater than or equal to value
<code>@Max(value)</code>	Number must be less than or equal to value
<code>@Email</code>	Must be a valid email
<code>@Pattern(regex)</code>	Must match the given regex

EXAMPLE

```
public class UserRequest {
    @NotBlank(message = "Name is required")
    private String name;
    @Email(message = "Invalid email")
    private String email;
    @Min(value = 18, message = "Age must be >= 18")
    private int age;
    @Size(min = 10, max = 10, message = "Mobile must be 10 digits")
    private String mobile;
}

@PostMapping("/user")
public ResponseEntity<String> createUser(@Valid @RequestBody UserRequest request) {
    return ResponseEntity.ok("User created successfully");
}
```

CUSTOM ERROR RESPONSE

```
@RestControllerAdvice
public class GlobalExceptionHandler {
    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ResponseEntity<Map<String, String>> handleValidationException(
        MethodArgumentNotValidException ex) {
        Map<String, String> errors = new HashMap<>();
        ex.getBindingResult().getFieldErrors().forEach(error ->
            errors.put(error.getField(), error.getDefaultMessage()));
        return ResponseEntity.badRequest().body(errors);
    }
}
```

SAMPLE ERROR RESPONSE (JSON)

```
{
  "name" : "Name is required",
  "email" : "Invalid email",
  "age" : "Age must be >= 18",
  "mobile" : "Mobile must be 10 digits"
}
```

BENEFITS

- Ensures data integrity.
- Reduces manual validation code.
- Provides meaningful error messages.
- Improves API reliability.
- Easy to customize and extend.

* Always validate at the boundary (Controller) level, never in Service.

≡ SPRING BOOT HANDBOOK ≡

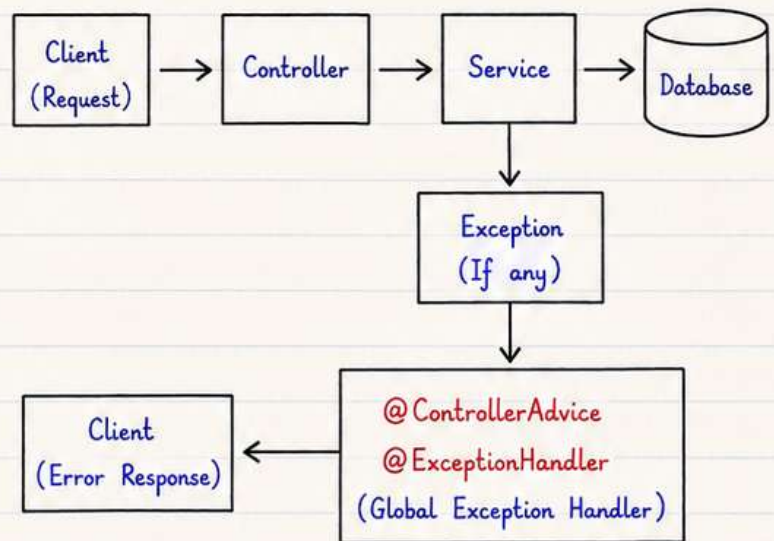
PART - 2

Page - 6/11

6. EXCEPTION HANDLING

HOW IT WORKS ?

- Exception handling helps us handle runtime errors in a clean and controlled way.
- Spring Boot provides `@ControllerAdvice` and `@ExceptionHandler` to handle exceptions globally.
- We can return meaningful error messages with proper HTTP status codes.



EXAMPLE - GLOBAL EXCEPTION HANDLER

`@ControllerAdvice`

```
public class GlobalExceptionHandler {  
  
    @ExceptionHandler(ResourceNotFoundException.class)  
    public ResponseEntity<String> handleNotFound(ResourceNotFoundException ex) {  
        return new ResponseEntity<>(ex.getMessage(), HttpStatus.NOT_FOUND);  
    }  
  
    @ExceptionHandler(Exception.class)  
    public ResponseEntity<String> handleAll(Exception ex) {  
        return new ResponseEntity<>("Something went wrong!",  
                                    HttpStatus.INTERNAL_SERVER_ERROR);  
    }  
}
```

SAMPLE ERROR RESPONSE

```
{  
  "timestamp" : "2025-06-18T10:30:00",  
  "status" : 404,  
  "error" : "Not Found",  
  "message" : "Resource not found",  
  "path" : "/api/users/10"  
}
```

TYPES OF EXCEPTIONS

Type	Description	Example	When it Occurs
Checked Exception	Checked at compile time. Must be handled.	IOException, SQLException	Database connection, File operations
Unchecked Exception	Not checked at compile time. Occurs at runtime.	NullPointerException, ArithmeticException	Null reference, Divide by zero
Custom Exception	User-defined exceptions for business scenarios.	ResourceNotFoundException, InvalidDataException	When specific business rules are violated

BEST PRACTICES

- Use `@ControllerAdvice` for global exception handling.
- Return meaningful messages with proper HTTP status codes.
- Log the actual exception details (don't expose to user).
- Create custom exceptions for business logic errors.

* NOTE:

A good exception handling improves user experience and makes debugging easier.

≡ SPRING BOOT HANDBOOK ≡

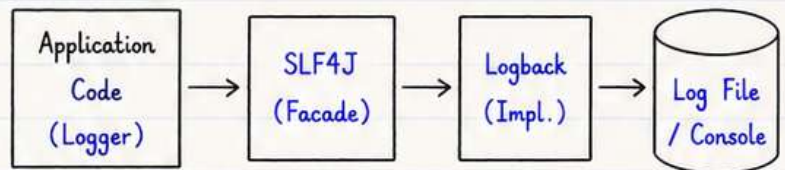
PART - 2

Page - 7/11

7. LOGGING

- Logging is essential for monitoring, debugging and maintaining applications.
- Spring Boot uses SLF4J as the logging facade and Logback as the default implementation.
- We can configure logging levels, formats and output destinations.

LOGGING FLOW



LOG LEVELS (IN ORDER)

TRACE < DEBUG < INFO < WARN < ERROR

(Most Detailed)

(Most Severe)

EXAMPLE - USING LOGGER IN CODE

```
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;

@RestController
public class UserController {
    private static final Logger log =
        LoggerFactory.getLogger(UserController.class);

    @GetMapping("/users")
    public List<String> getUsers() {
        log.info("Fetching all users");
        log.debug("Request received for /users");
        log.warn("No users found in database");
        log.error("Error while fetching users", new RuntimeException());
        return List.of("Aman", "Rohit");
    }
}
```

LOG LEVEL MEANING

Level	Meaning
TRACE	Very detailed information, used for deep debugging.
DEBUG	Detailed debug information.
INFO	General information about application flow.
WARN	Something unexpected happened but application is still running.
ERROR	Serious problem, application might fail to perform some operations.

DEFAULT LOG PATTERN (LOGBACK)

```
%d{yyyy-MM-dd HH:mm:ss.SSS} - %level - %logger
[%thread] - %msg%n
```

%d → Date & Time
%level → Log Level
%logger → Logger Name
%thread → Thread Name
%msg → Log Message

CONFIGURATION - application.properties

```
# Set root logging level
logging.level.root=INFO

# Set logging level for specific package
logging.level.com.example.demo=DEBUG

# Log file location
logging.file.name=logs/app.log

# Pattern for console logs
logging.pattern.console=%d{yyyy-MM-dd HH:mm:ss} - %level - %msg%n
```

OUTPUT EXAMPLE (CONSOLE)

```
2025-06-18 10:15:30 - INFO - com.example.demo.UserController
[http-nio-8080-exec-1] - Fetching all users
2025-06-18 10:15:30 - DEBUG - com.example.demo.UserController
[http-nio-8080-exec-1] - Request received for /users
2025-06-18 10:15:31 - WARN - com.example.demo.UserController
[http-nio-8080-exec-1] - No users found in database
2025-06-18 10:15:31 - ERROR - com.example.demo.UserController
[http-nio-8080-exec-1] - Error while fetching users
```

BENEFITS

- Helps in debugging and troubleshooting.
- Provides runtime visibility.
- Useful in production for monitoring issues.
- Can be easily configured without code changes.

* NOTE:

Always use appropriate log levels.
Avoid using `System.out.println()` in production code.

≡ SPRING BOOT HANDBOOK ≡

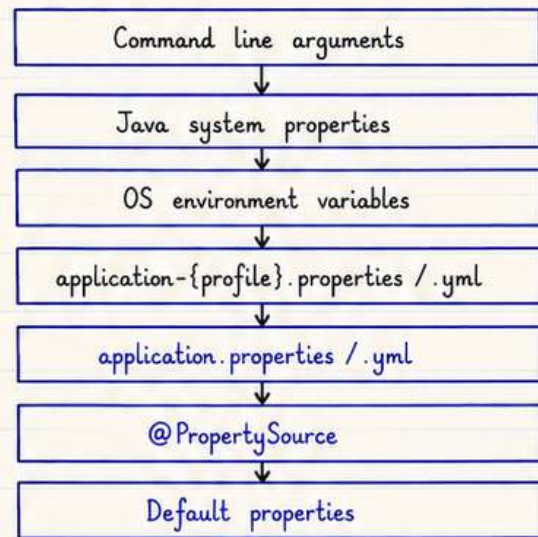
PART - 2

Page - 8/11

8. CONFIGURATION

- Spring Boot externalizes configuration using `application.properties` or `application.yml` files.
- It helps us to separate configuration from code.
- We can access these properties in our application using `@Value`, `@ConfigurationProperties` etc.
- Different environments can have different configuration.

CONFIGURATION LOADING ORDER (HIGH TO LOW)



PROPERTIES FILE (`application.properties`)

```
server.port=8080
spring.application.name=demo-app
spring.datasource.url=jdbc:mysql://localhost:3306/demo
spring.datasource.username=root
spring.datasource.password=root
logging.level.org.springframework=INFO
logging.file.name=logs/app.log
```

YAML FILE (`application.yml`)

```
server:
  port: 8080
spring:
  application:
    name: demo-app
  datasource:
    url: jdbc:mysql://localhost:3306/demo
    username: root
    password: root
  logging:
    level:
      org.springframework: INFO
  file:
    name: logs/app.log
```

ACCESSING PROPERTIES IN CODE

```
// Using @Value
@Value("${app.name}")
private String appName;

// Using @ConfigurationProperties
@Component
@ConfigurationProperties(prefix = "app")
public class AppProperties {
    private String name;
    private String author;
    // getters and setters
}
```

TYPES OF PROPERTIES

Type	Description	Example
Simple Property	Key-value pair	<code>server.port=8080</code>
Nested Property	Hierarchical structure	<code>spring.datasource.url=...</code>
List Property	List of values	<code>app.items=a,b,c</code>
Map Property	Key-value in map	<code>app.map.key1=value1</code>

BEST PRACTICES

- Keep sensitive data in environment variables.
- Use meaningful property names.
- Use profiles for environment-specific configurations.
- Keep default values in `application.properties`.
- Never commit real credentials in code repository.

* NOTE:

Externalized configuration makes application flexible, maintainable and environment independent.

≡ SPRING BOOT HANDBOOK ≡

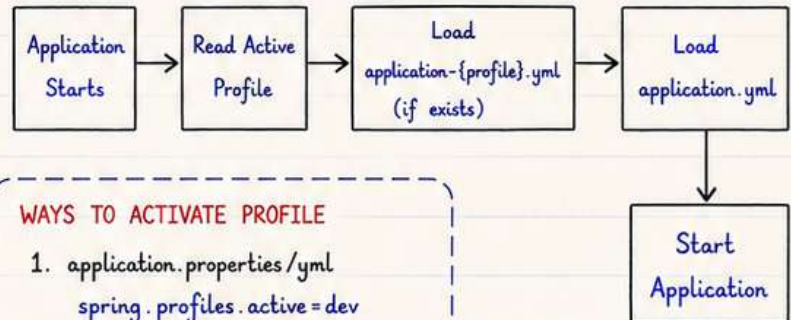
PART - 2

Page - 9/11

9. PROFILE & ENVIRONMENT

- Profiles allow us to define different configurations for different environments like dev, test, prod etc.
- Spring Boot automatically picks the appropriate profile based on the active setting.
- We can define environment specific properties using `application-{profile}.yml`
- Profiles help in separating configuration and making deployments easy.

HOW PROFILES WORK ?



WAYS TO ACTIVATE PROFILE

1. `application.properties/yml`
`spring.profiles.active=dev`
2. Command Line
`java -jar app.jar --spring.profiles.active=dev`
3. Environment Variable
`SPRING_PROFILES_ACTIVE=dev`

EXAMPLE STRUCTURE

```
src
├── main
│   └── resources
│       ├── application.yml      (common config)
│       ├── application-dev.yml  (dev config)
│       ├── application-test.yml (test config)
│       └── application-prod.yml (prod config)
```

SAMPLE FILES

```
application.yml
server:
  port: 8080
spring:
  profiles:
    active: dev
app:
  name: MyApp
```

```
application-dev.yml
server:
  port: 8081
db:
  url: jdbc:mysql://localhost:3306/devdb
  username: dev
  password: dev
```

```
application-prod.yml
server:
  port: 80
db:
  url: jdbc:mysql://prod-db:3306/proddb
  username: prod
  password: prod
```

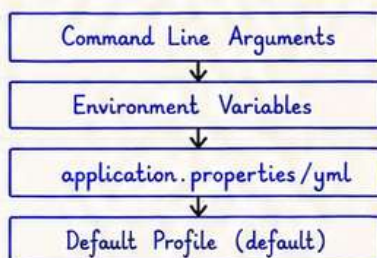
@PROFILE ANNOTATION

```
@Component
@Profile("dev")
public class DevOnlyService {
    // This bean will be loaded only in dev profile
}
```

MULTIPLE PROFILES

```
@Profile({"dev", "test"})
public class DevOrTestService {
    // This bean will be loaded in dev or test profiles
}
```

PROFILE PRIORITY (HIGH TO LOW)



BUILT-IN PROFILES

Profile	Activated When	Use Case
dev	<code>spring.profiles.active=dev</code>	Local development
test	<code>spring.profiles.active=test</code>	Testing environment
prod	<code>spring.profiles.active=prod</code>	Production environment
default	No profile is set	Fallback configuration

BEST PRACTICES

- Keep common properties in `application.yml`.
- Store sensitive data in environment variables.
- Use meaningful profile names.
- Avoid hardcoding environment specific values in code.
- Test your application with each profile before deployment.

* NOTE :

Profiles are powerful in managing different environments with same codebase.
Use them wisely for clean and scalable applications.

≡ SPRING BOOT HANDBOOK ≡

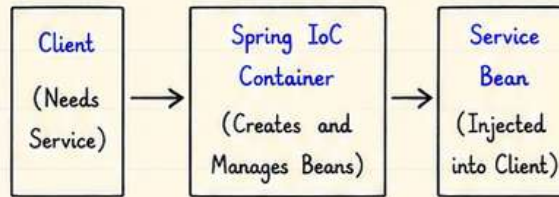
PART - 2

Page - 10/11

10. DEPENDENCY INJECTION

- Dependency Injection (DI) is a core feature of Spring.
- It allows us to inject the dependencies into a bean rather than creating them manually.
- Spring provides different ways to perform DI.
- DI makes our code loosely coupled, easy to test and maintain.

HOW DI WORKS ?



Container **creates** objects, **manages** their lifecycle and **injects** wherever required.

TYPES OF DEPENDENCY INJECTION

Type	Description	Example
Constructor Injection	Dependencies passed via constructor.	@Autowired (on constructor)
Setter Injection	Dependencies set via setter method.	@Autowired (on setter method)
Field Injection	Dependencies injected into fields.	@Autowired (on field)

EXAMPLE - CONSTRUCTOR INJECTION

```

@Service
public class UserService {
    private final UserRepository userRepository;

    @Autowired
    public UserService(UserRepository userRepository) {
        this.userRepository = userRepository;
    }
}
  
```

EXAMPLE - SETTER INJECTION

```

@Service
public class UserService {
    private UserRepository userRepository;

    @Autowired
    public void setUserRepository(UserRepository userRepository) {
        this.userRepository = userRepository;
    }
}
  
```

EXAMPLE - FIELD INJECTION

```

@Service
public class UserService {
    @Autowired
    private UserRepository userRepository;
}
  
```

BEAN ANNOTATIONS (COMMON)

Annotation	Use
@Component	Generic stereotype for any Spring bean.
@Service	For service layer components.
@Repository	For data access layer components.
@Controller	For controller layer components.
@Configuration	Marks a class as source of bean definitions.

BEAN SCOPE

Scope	Description	Use Case
singleton	One bean instance per Spring container (default).	Default scope for most beans.
prototype	New instance created for every request.	When each request needs a fresh instance.
request	One bean per HTTP request.	Web applications.
session	One bean per HTTP session.	Store user specific data.
application	One bean per ServletContext.	Web application wide data.
websocket	One bean per WebSocket session.	WebSocket applications.

QUALIFIERS

```

@Service
@Qualifier("emailService")
public class EmailService implements NotificationService {
    public void send() {
        System.out.println("Email Sent");
    }
}

@Autowired
@Qualifier("emailService")
private NotificationService notificationService;
  
```

BENEFITS OF DI

- Reduces tight coupling.
- Improves testability (Easy to mock dependencies).
- Promotes clean and maintainable code.
- Makes the application scalable and flexible.

* NOTE :

Always prefer Constructor Injection for mandatory dependencies as it ensures immutability and helps in writing robust and testable code.

≡ SPRING BOOT HANDBOOK ≡

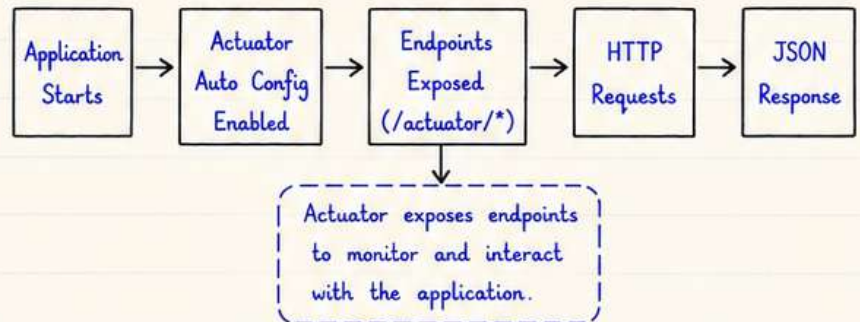
PART - 2

Page - 11/11

11. SPRING BOOT ACTUATOR

- Spring Boot Actuator helps us monitor and manage our application in production.
- It provides production-ready features to check health, metrics, info, env, logs etc.
- It exposes HTTP endpoints that can be accessed remotely.
- It is very useful for DevOps, monitoring and troubleshooting.

HOW ACTUATOR WORKS ?



COMMON ENDPOINTS

Endpoint	Description
<code>/actuator/health</code>	Application health status
<code>/actuator/info</code>	Application information
<code>/actuator/metrics</code>	Application metrics
<code>/actuator/env</code>	Environment properties
<code>/actuator/loggers</code>	View & change log levels
<code>/actuator/mappings</code>	List all request mappings
<code>/actuator/heapdump</code>	Get thread dump

ACTUATOR DEPENDENCY (MAVEN)

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-actuator</artifactId>
</dependency>
```

ENABLE ENDPOINTS

```
management:
  endpoints:
    web:
      exposure:
        include: "*" # expose all endpoints
```

CUSTOM PORT FOR ACTUATOR

```
management:
  server:
    port: 9090
# Actuator will run on port 9090
```

SECURITY FOR ACTUATOR

```
management:
  endpoints:
    web:
      exposure:
        include: health, info
spring:
  security:
    user:
      name: admin
      password: admin123
```

EXAMPLE - HEALTH ENDPOINT OUTPUT

```
{
  "status": "UP",
  "components": {
    "db": { "status": "UP" },
    "diskSpace": { "status": "UP" }
  }
}
```

EXAMPLE - APPLICATION.PROPERTIES

```
management.endpoints.web.exposure.include=*
management.endpoint.health.show-details=always
management.info.env.enabled=true
info.app.name=My Spring Boot App
info.app.version=1.0.0
```

ACTUATOR BENEFITS

- Helps in health check and monitoring.
- Provides real-time metrics.
- Can be integrated with monitoring tools (Prometheus, Grafana, etc.).
- Helps in debugging and troubleshooting.
- Easy to enable and use.

* NOTE :

Always secure actuator endpoints in production.
Never expose all endpoints without authentication.
Use only required endpoints for monitoring.

* KEEP LEARNING. BUILD PROJECTS. STAY CURIOUS. GROW EVERYDAY. *

♡ ≡ FOLLOW ≡ ♡
→ FOR ←
≡ PART 3 ≡

MORE ADVANCED TOPICS,
EXAMPLES & REAL-WORLD PROJECTS AHEAD!

≡ COMMENT ≡ ☆
≡ SPRING BOOT ≡

↓
FOR PDF NOTES



☆ ≡ FOR PART 3 ≡ ☆
TARGET IS 5K ❤️